



Human authority
before irreversible action.

Aegis Genesis

Radar Collision Avoidance

Human-Governed Decision Architecture for Safer Autonomous Space Traffic

Edgar Fu | Carnegie Mellon University

40 in W × 44 in H

WHAT SETS AEGIS GENESIS APART

Aegis Genesis links collision-risk equations, an explicit human-review rule, executable C++ and boundary tests in one traceable demonstrator.

Human involvement in collision avoidance is not new. Aegis Genesis focuses on a testable, traceable boundary between machine recommendation and human authority; authorization enforcement and maneuver execution remain proposed.

Background & Motivation

1 / WHY - ORBITAL CROWDING AND COLLISION RISK

Active satellites (communication, Earth observation, navigation), inactive spacecraft, spent rocket bodies and collision debris crowd shared orbits. More conjunctions increase decision complexity.

Consequences: thousands of debris fragments and cascading orbital risk; satellite/shared-orbit disruption; future-mission risk; accountable, traceable decisions are needed.

Before AI acts, who has the authority? Human involvement exists; Aegis formalizes the decision boundary.

Project Objectives

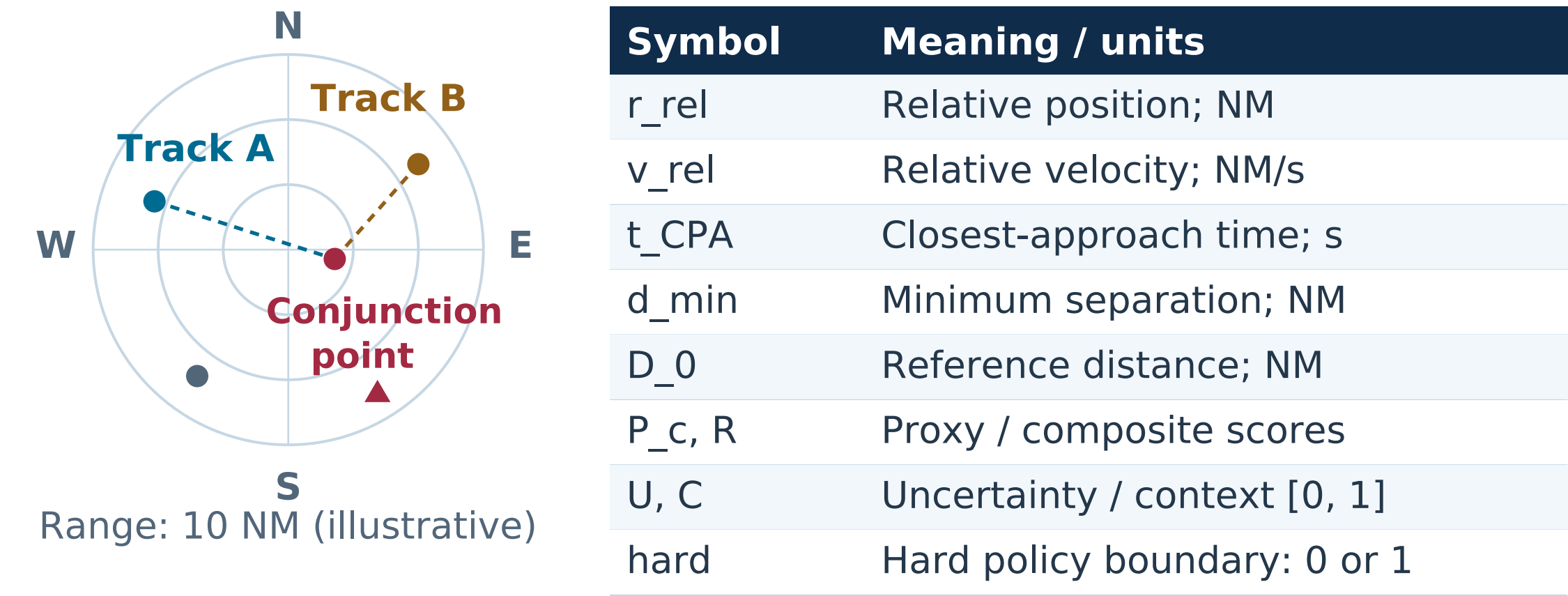
- Verify relative-motion calculations in executable C++.
- Make human-review thresholds and source-to-result links explicit.
- Keep irreversible action under human authority for safer, sustainable orbital traffic.

Project Details / Description

Instrumentation / Modules

2 / SEE - OBSERVATION AND RADAR INPUTS

Illustrative radar only. The code uses supplied relative-motion inputs; no radar hardware or real space-situational-awareness (SSA) data.



A/B: active tracks. Gray: inactive object. Red triangle: debris. Dashed paths: predicted trajectories in shared orbits.

Data: supplied $r/v \rightarrow CPA \rightarrow separation \rightarrow proxy \rightarrow R \rightarrow H$. Evidence recording and replay are proposed.

3 / CALCULATE - MATHEMATICAL CORE

Relative position / velocity
 $r_{rel} = r_A - r_B$; $v_{rel} = v_A - v_B$

Closest approach
 $t_{CPA} = -(r_{rel} \cdot v_{rel}) / \|v_{rel}\|^2$

Minimum separation
 $d_{min} = \|r_{rel} + t_{CPA} v_{rel}\|$

Illustrative risk proxy
 $P_c = \exp[-(d_{min} / D_0)^2]$

Composite risk
 $R = 0.60 P_c + 0.20 U + 0.20 C$

Human-review trigger
 $H = 1$ if $R > 0.700$ or $t_{CPA} < 120$ s or $hard = 1$

Nominal example
 $r = (1.00, 0.32)$ NM; $v = (-0.010, 0)$ NM/s; $D_0 = 1.00$ NM
 $U = 0.50$; $C = 0.90$; $hard = 0$; weights = 0.60, 0.20, 0.20
 $t_{CPA} = 100.0$ s; $d_{min} = 0.320$ NM; $P_c = 0.903$; $R = 0.822$; $H = 1$

H = 1 requires review; it is not permission to execute. P_c is dimensionless and illustrative, not a measured 90.3% collision probability.

5 / DECIDE - PROPOSED HUMAN AUTHORITY GATE

Proposed human review board: **safety, dynamics, engineering, mission and program** roles independently verify the risk model and review all recommendations before final authorization.

Decisions: Approve Burn · Hold for More Tracks · Decline Burn · Escalate Review. Reviewers may modify plans.

6 / GOVERN - PROPOSED CLOSED-LOOP WORKFLOW

- Execute the authorized maneuver plan.
- Verify the post-maneuver outcome.
- Record evidence, decisions and results.
- Feed validated outcomes into governed updates.
- Return to autonomous monitoring.

Cycle: Sense → Decide → Execute → Verify → Learn → Monitor.

Illustrative outcomes: Evidence YES · Replay YES · Authority YES · Audit YES.

Human intervention stays within this proposed loop. Authorization enforcement, maneuver execution, logs and replay remain future work.

Design & Build Process

4 / VALIDATE - COMPLETE VERIFIED C++17 SOURCE

75 source lines, wrapped for display. Original .cpp embedded in this PDF.

```
1 #include <cmath>
2 #include <iomanip>
3 #include <iostream>
4 #include <stdexcept>
5 #include <string>
6 struct Vec2 { double x, y; };
7 void requireFinite(double x) { if (!std::isfinite(x)) throw
    std::invalid_argument("Nonfinite input"); }
8 void score(double x) { requireFinite(x); if (x < 0 || x > 1) throw
    std::invalid_argument("Score outside [0,1]"); }
9 double timeToCPA(Vec2 r, Vec2 v) {
10     requireFinite(r.x); requireFinite(r.y); requireFinite(v.x); requireFinite(v.y);
11     const double vv = v.x*v.x + v.y*v.y;
12     if (!std::isfinite(vv) || vv <= 0) throw
        std::invalid_argument("Undefined CPA time");
13     const double t = -(r.x*v.x + r.y*v.y)/vv;
14     requireFinite(t); return t;
15 }
16 double distanceAtCPA(Vec2 r, Vec2 v) {
17     const double t = timeToCPA(r,v);
18     const double d = std::hypot(r.x+v.x*t,r.y+v.y*t);
19     requireFinite(d); return d;
20 }
21 double collisionProxy(double dMin, double D0) {
22     requireFinite(dMin); requireFinite(D0);
23     if (dMin < 0 || D0 <= 0) throw
        std::invalid_argument("Invalid distance or scale");
24     const double ratio = dMin/D0;
25     return std::exp(-ratio*ratio);
26 }
27 double compositeRisk(double Pc, double U, double C) {
28     score(Pc); score(U); score(C); return .60*Pc + .20*U + .20*C;
29 }
30 bool humanReviewRequired(double R, double tCPA, bool hardBoundary) {
31     score(R); requireFinite(tCPA); return R > .70 || tCPA < 120.0 || hardBoundary;
32 }
33 // Deliberate Task 16 fault, reachable only with --demonstrate-bug.
34 // The production function timeToCPA above retains the correct minus sign.
35 double wrongTimeToCPA(Vec2 r, Vec2 v) {
36     return (r.x*v.x+r.y*v.y)/(v.x*v.x+v.y*v.y);
37 }
38 int main(int argc, char** argv) {
39     const Vec2 r{1.00, 32}, v{-0.010, 0};
40     std::cout << std::fixed << std::setprecision(12);
41     if (argc == 2 && std::string(argv[1]) == "--demonstrate-bug") {
42         const double t=wrongTimeToCPA(r,v);
43         std::cout << "WRONG t_CPA = " << t << " s\n";
44         << std::hypot(r.x+v.x*t,r.y+v.y*t) << " NM\n";
45         return 0;
46     }
47     if (argc != 1) { std::cerr << "Usage: aegis_genesis [--demonstrate-bug]\n";
48         return 2; }
49     const double pc=timeToCPA(r,v), d=distanceAtCPA(r,v);
50     const double pc=collisionProxy(d,1.0), risk=compositeRisk(pc,.50,.90);
51     const bool hh=humanReviewRequired(risk,t,false);
52     std::cout << "t_CPA = " << t << " s\n";
53     << "\nR = " << risk << "\nH = " << hh << "\nT_margin = " << t-45 <<
54     " s\n";
55     int passed=0, total=0;
56     auto check=[&](const std::string& name, bool ok) {
57         ++total; passed += ok ? 1 : 0;
58         std::cout << (ok ? "PASS " : "FAIL ") << name << '\n';
59     };
60     auto near=[](double a,double b){return std::abs(a-b)<=1e-9;};
61     check("nominal approach", near(t,100)&&near(d,.32)&&near(pc,.90268412080421)&&near(risk,.8216010472485652)&&hh);
62     check("exact risk threshold: R=.700 t=120 hard=0 expected H=0",
63         !humanReviewRequired(.700,120,false));
64     check("just above risk: R=.701 t=120 hard=0 expected H=1",
65         humanReviewRequired(.701,120,false));
66     check("time threshold equality: R=.650 t=120 hard=0 expected H=0",
67         !humanReviewRequired(.650,120,false));
68     check("time threshold crossing: R=.650 t=119 hard=0 expected H=1",
69         humanReviewRequired(.650,119,false));
70     check("hard boundary: R=.200 t=300 hard=1 expected H=1",
71         humanReviewRequired(.200,300,true));
72     check("low risk control: R=.200 t=300 hard=0 expected H=0",
73         !humanReviewRequired(.200,300,false));
74     bool rejected=false;
75     try { (void)timeToCPA(r,{0,0}); } catch(const std::invalid_argument&){
76         {rejected=true;}
77         check("zero relative velocity rejected",rejected);
78     }
79     try { (void)collisionProxy(.32,0); } catch(const std::invalid_argument&){
80         {rejected=true;}
81         check("zero reference distance rejected",rejected);
82     }
83     check("receding motion retains workbook signed time",near(timeToCPA(r,{0.010,0}),
84         -100));
85     std::cout << "Tests: " << passed << '/' << total << " passed\n";
86     return passed==total ? 0 : 1;
87 }
```

SHA-256: complete original source, aegis_genesis.cpp
ee20c87d65b9fd3915ee7ef8997c2d11
6078e1466746c584cc988df7511d9355

Diagram / Build Photos

SYSTEM ARCHITECTURE - CALCULATIONS TO AUTHORITY



Current code: supplied inputs → numerical risk and H.

Proposed workflow: $H = 1 \rightarrow$ human review; $H = 0 \rightarrow$ continued monitoring. Human authorization is required before irreversible execution.

Math-to-code mapping: $t_{CPA} \leftrightarrow \text{timeToCPA}()$; $d_{min} \leftrightarrow \text{distanceAtCPA}()$; $P_c \leftrightarrow \text{collisionProxy}()$; $R \leftrightarrow \text{compositeRisk}()$; $H \leftrightarrow \text{humanReviewRequired}()$.

Challenges We Solved

CPA sign: fault injection exposes a physically misleading result.

Boundaries: equality, crossing and hard-policy tests check strict comparisons.

Invalid inputs: reject zero relative velocity and zero D_0 .

Traceability: signed receding time, source hash and actual outputs connect equations to evidence.

Results & Discussions / Impact

BUILD & RUN VERIFICATION - ACTUAL OUTPUT

Source: aegis_genesis.cpp · **Environment:** Linux x86_64; g++ 13.3.0
g++ -std=c++17 -O2 -Wall -Wextra -Wpedantic -Werror
aegis_genesis.cpp -o aegis_genesis

Build exit code: **0** · Compiler output: **none** · Run: **./aegis_genesis**

$t_{CPA} = 100.000000000000$ s $d_{min} = 0.320000000000$ NM
 $P_c = 0.90268412081$ $R = 0.821601047249$
 $H = 1$ $T_{margin} = 55.000000000000$ s

Tests: 10/10 passed · Run exit code: 0. $T_{margin} = 100 - 45 = 55$ s
assumes an illustrative 45-second response budget.

Test case / inputs	Expected	Result
1 Nominal approach	H = 1	PASS
2 R=.700; t=120; hard=0	H = 0	PASS
3 R=.701; t=120; hard=0	H = 1	PASS
4 R=.650; t=120; hard=0	H = 0	PASS
5 R=.650; t=119; hard=0	H = 1	PASS
6 R=.200; t=300; hard=1	H = 1	PASS
7 R=.200; t=300; hard=0	H = 0	PASS
8 Zero relative velocity	Reject	PASS
9 Zero reference distance	Reject	PASS
10 Receding motion	t=-100 s	PASS

Times in seconds; tolerance 10^{-9} ; Boolean checks exact. Nominal test checks geometry, proxy and risk.

FAULT INJECTION - TASK 16

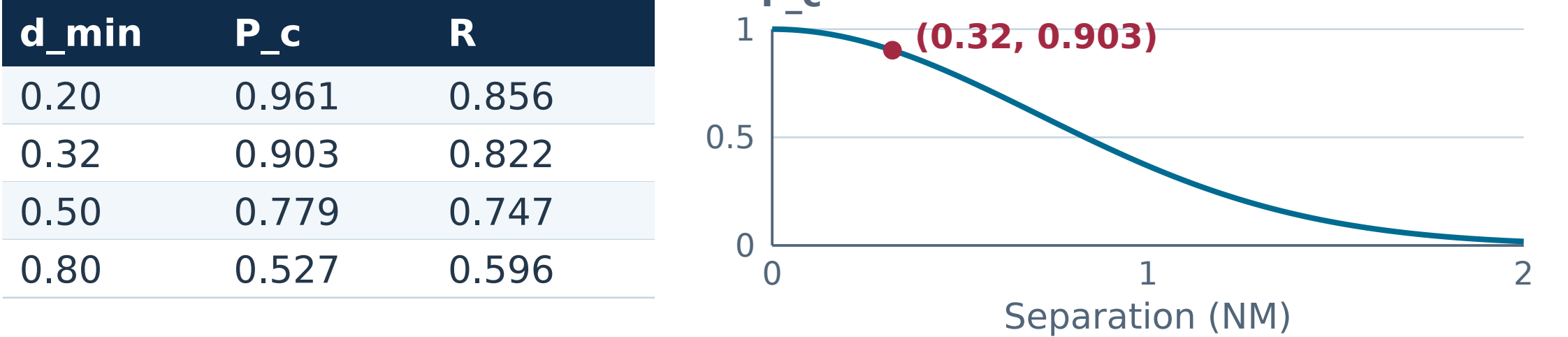
./aegis_genesis --demonstrate-bug

WRONG $t_{CPA} = -100.000000000000$ s

WRONG separation = 2.025438224188 NM

Exit code: 0. The fault is isolated from the correct production CPA.

SENSITIVITY & RISK PROXY VS. SEPARATION



$D_0 = 1.00$ NM; $U = 0.50$; $C = 0.90$. Illustrative risk proxy.

OPTION COMPARISON - ILLUSTRATIVE

Option	Av*	Residual	Fuel	Mission	Result
A	0.42	0.02%	Low	Low	Recommended
B	0.31	0.08%	Low	Low	Acceptable
C	0.15	0.20%	Very low	Medium	Marginal
D	0.00	1.20%	None	None	Unsafe

*Av units unspecified. Options and residual-risk percentages are illustrative; the program does not generate them.

GOVERNANCE STATUS - ILLUSTRATIVE EXAMPLE

Field	Value	Field	Value
Decision status	Authorized	Execution	Completed
Post-maneuver risk	0.003	New t_{CPA}	>10,000 s
New d_{min}	12.4 NM	Log ID	AG-2026-0427
Archive	Stored	Audit ready	Yes

Status, risk 0.003 and YES outcomes illustrate the proposed workflow; not program outputs. Post-maneuver risk metric unspecified.

Conclusion & Next Steps

7 / CONTRIBUTIONS, LIMITATIONS & FUTURE DIRECTIONS

Contributions: explicit, testable human-review gate; integrated risk/review logic; traceable C++ with 10/10 tests; uncertainty and consequence scores; human-governed design for safer autonomous operations.

Limits: simplified 2D constant velocity; illustrative proxy; no real SSA data or environmental model; no maneuver optimizer or real fuel model; academic demonstrator. Tests validate calculations and review logic, not operational flight performance.

Future: realistic 3D orbital propagation; covariance-based collision probability; real SSA/multi-object tracking; human-governed maneuver optimization; aviation, robotics and critical infrastructure. Implement authorization enforcement, logs and replay.

AI recommends; human authority approves irreversible action.

A Story Worth Telling

One minus sign changed CPA (+100 to -100 s) and separation (0.320 to 2.025 NM). Reviewers need equations and code to agree.

Acknowledgments

Event organizers and the supplied template. AI supported code and poster preparation; outputs were verified by execution.